

ADR-001: Contentstack Demonstrations Portal UI

Status: Proposed

Date: October 03, 2025

Context

The Sales Engineering, Solutions Architect, Partners, and Demo Engineering teams currently lack a single, centralized repository to easily find demonstration materials. At present, demo instructions, recordings, and related resources are scattered across Confluence and Google Drive, making discovery inefficient and collaboration fragmented.

We need to build a portal where all demo instructions and demo components are collected into a single repository and the interface for this portal be easy, and intuitive to use. Users should have no problem navigating the portal and finding what they need to find.

Key Non-Functional Requirements (NFRs)

- Appearance: Customizable UI themes (light and dark mode) and styles (icons and text theming) clean and modern design
- Convenience: Live search results in the search bar in the navigation bar that link to whatever you're searching for
- Speed: The UI should respond to user input within a fast interval of time
- Overall accessibility: The UI should be customizable and configurable for users, so users with disabilities could change the navigation to make the portal easier to use.

Options

1. React JS:
 - a. Pros: Built around reusable components making the code very maintainable and modular, efficient in updating the UI making performance generally fast
 - b. Cons: Server-side rendering requires extra setup and configuration
2. Next.js (chosen):
 - a. Pros: Builds upon React whilst adding more features, API endpoints are consolidated into one project
 - b. Cons: A less flexible than React with a bit of a steeper learning curve
3. Angular:
 - a. Pros: Very scalable and plentiful with built-in features
 - b. Cons: Restrictive for customized architectures, steeper learning curve

Decision

We will pursue Option 2: Next.JS to build our UI from. With Next's added features on top of the React framework and being overall more simpler to users than Angular, we believe that this option would be the best one going forward. With this technology being quicker to onboard than Angular and being more robust than React, this will meet our needs in constructing a dynamic, easy to navigate UI in a reasonable amount of time and could be improved upon overtime with the added features.

Consequences

Positive:

- Quicker to onboard and learn than Angular
- More feature-robust than React
- Less dependant on third party libraries

Negative:

- Slight learning curve and stricter structure to code in
- Optimal performance is heavily dependent on Vercel's ecosystem

Risks and Mitigations

- #1: Long Load times
 - Risk: Due to its more robust functionality, the UI may take longer to load and build, reducing performance
 - Mitigation: Optimize the UI so that there isn't too much overhead in loading the UI. Don't over engineer the interface with too many unneeded features
- #2: The design of the UI
 - Risk: The UI will end up too confusing and users won't know how to use the portal
 - Mitigation: Do user testing and take suggestions from some on how to improve the UI and follow best practices in designing a clean, understandable UI
- #3: Low Adoption
 - Risk: Even if the UI is fully tested and given the go-ahead, some might not adjust well to it and will require assistance
 - Mitigation: Provide a brief guide on how to get started with the interface and emphasize the instructions for functionality users will use the most

References

Next.js documentation: <https://nextjs.org/>